SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO5 - Naturalization (BL5, BL6)	**Assessment:**	Assessment Tool 2 - Pseudocode Writing Task
Weightage:	30% of CIA	**Total Marks:**	100
CO5 Statement:	*Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.*		
Student Name:	CH.POORNA TEJA	**Reg. No.:**	192472149
Section / Batch:		**Date:**	

Instructions to Students

- Answer the following question with proper pseudocode and logical explanation.
- Clearly specify the algorithmic approach and complexity class wherever applicable.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) for complexity analysis.
- Include state-space representation, recursion steps, and pruning conditions in backtracking problems.
- Neat presentation and step-by-step justification are mandatory.

Question Type	Focus Area	Questions
Pseudocode Writing Task	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – Pseudocode Writing Task

1. Write a pseudocode to find whether a clique of size k exists in a graph using Backtracking. Clearly define inputs (graph, integer k) and output (true/false or clique set). Design an algorithm that generates vertex subsets and checks adjacency constraints. Use loops and recursion effectively to explore combinations. Apply pruning to eliminate subsets that cannot form a clique. Present the pseudocode in a structured format. Ensure correctness and completeness.

Code of Conduct

I P. Sai Charan Tej(192465048)__(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm Design	30%				
Use of Control Structures	20%				
Pseudocode Structure	10%				
Completeness	20%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Algorithm: Clique Detection Using Backtracking

1. **Start**
2. Read the number of vertices n and edges m.
3. Create an adjacency matrix for the graph.
4. Read all edges and mark the corresponding vertices as adjacent.
5. Read the required clique size k.
6. Initialize an empty list clique.
7. Start the **backtracking** function from the first vertex.
8. If the size of clique becomes k, return the clique.
9. For each remaining vertex:
    - Check whether it is adjacent to **every vertex** already in clique.
    - If yes, add the vertex to clique.
    - Recursively continue the search.
10. If the required clique cannot be formed with the current vertex, **remove it** from clique (backtrack).
11. **Prune** the branch if the number of remaining vertices is insufficient to reach size k.
12. Continue until all possible combinations are checked.
13. If a clique of size k is found, print the clique.
14. Otherwise, print **"Clique of size k does not exist."**
15. **Stop**.

Correctness

The algorithm adds a vertex only when it is adjacent to every vertex already selected. Therefore, every generated subset is a valid clique. When the clique reaches size k, it is guaranteed to contain k mutually adjacent vertices. Since the algorithm recursively considers all valid vertex combinations that are not eliminated by safe pruning, it will find a clique if one exists.

Complexity

The worst-case time complexity is **O(C(n,k) × k)**, where C(n,k) represents the number of possible subsets of k vertices. The problem is computationally difficult in general and is related to the **NP-complete Clique Decision Problem**.

Output:

Q1 **Graph Coloring Problem for Minimum Color Usage** 100 marks

Write a pseudocode using Backtracking to determine the minimum number of colors required to color a graph. Clearly define inputs (graph) and output (minimum color count and valid coloring). Design an algorithm that tries coloring the graph with increasing numbers of colors until a valid assignment is found. Use loops and recursive coloring logic effectively for systematic exploration. Apply pruning whenever a color assignment violates adjacency constraints. Present the pseudocode in a neat and logically organized format. Ensure the solution handles all graphs correctly and produces the minimum feasible color count.

Your Code

```
def is_safe(vertex, color, graph, colors):
    for i in range(len(graph)):
        if graph[vertex][i] == 1:
            if colors[i] == color:
                return False
    return True


def graph_coloring(vertex, max_colors, graph, colors):
    if vertex == len(graph):
        return True

    for c in range(1, max_colors + 1):
        if is_safe(vertex, c, graph, colors):
            colors[vertex] = c
```

Input

```
4
0 1 1 0
1 0 1 1
1 1 0 1
```

Output

```
Enter number of vertices: Enter
adjacency matrix:

Minimum colors = 3
Coloring = [1, 2, 3, 1]
```